THESIS LOG

RILEY WOOD

1. Week 2

1.1. **What I learned this week.** I finally got a good sense of how the probabilistic polyhedra model work, mostly through conversation with Greg and writing a definition for it based on TOPAZ. I also spent some time reading [1] which gives some important information about how the probabilistic polyhedra domain works. The central algorithm of the paper is not something I am super comfortable with, but I learned from Manu that it is how the probabilistic polyhedra domain handles intersections, and the idea of taking intervals in a "generic position" seems really important.

I think I properly identified what is actually hard about the widening operation for the first time. The main issue seems to be that we need a widening "up to affine isomorphism" which requires some choices to be made before using traditional widenings.

1.2. **What I wrote this week.** I wrote definitions for program state, concrete lattice, probabilistic polyhedra which I think are pretty workable as definitions I also spent some time trying to define concrete and abstract semantics, but I confused myself a little and these are not good. The concrete semantics definition I did manage to fix, but abstract definitions still need to be fixed. I also stole a definition of Zonotopes from TOPZ (Which I don't really know how to cite) and [2].

1.3. **Questions.**

- What does it mean for polyhedra to be in generic position?
- Is it useful to split a "central polyhedron" out from the noise symbol constraints that get introduced during runtime?
- How much does the noise symbol polyhedron get affinely transformed in the existing implementation?

1.4. **Next week plan.** It seems like the next step is to answer the generic position question, and then work on an algorithm of the form (1) move noise symbol polyhedron to generic position (2) widen noise symbol polyhedron (3) widen variable polyhedra. Writing-wise, I can at the very least finish fixing all of my definitions. I think I know what needs to change now, I just didn't figure it out until after most of my work time for the week.

Date: October 8, 2026.

2. Week 2

2.1. **What I learned this week.** Reviewed Kripke structures, abstract interpretation, our favorite Galois connection, and widening operators (in the general sense). Overall not a great week for new content, I spent a lot of time on reminding myself what is even going on with abstract interpretation.

2.2. **What I wrote this week.** A bit of an introduction to formal verification. I don't think it's very good, but it should give me a framework to build upon down the line. Maybe struggling with precision when writing this, since I want it to be really accessible. It might be better in the future to start with definitions and formalism, then build narrative around that.

2.3. **Questions.**

- Is my notion of program state correct? (no one seems to bother formalizing this anywhere)
- What is going on with the zonotope definition that [2] gives?
- what is the two variables per inequality domain?
- What is going on with the approximations in Greg's paper? (Riemann approximations. Can we do better for polytopes?)
- What is there to be said about Rice's theorem?
- Kleene algebras show up when talking about methods for building state diagrams. What are these?

2.4. **Next week plan.** I think that this week I may have been too focused on writing narrative and studying fundamentals. Going forward maybe focus on more on formalism, and build narrative later.

Revisiting ToPAZ pretty closely might be worthwhile, but I have spent a fair bit of time with it already.

I will spend a lot of time on [3]. 34 pages on how to do polyhedra widenings! Yes please. This article seems like a good central point to do some research. It cites some things I know will be important (Chernikova!) and gets cited fairly frequently (recently by a paper on selective widening and two variables per inequality)

3. Week 1

3.1. **What I learned this week.**

- Read [4], a paper looking to use algebraic effects and domain theory to give a calculus and abstract machine for functional logic programming. They base this off of **Call By Push Value** (CBPV). It is a little more domain theoretic than I maybe am planning to get with this project, but it does fit into a pattern of papers from this year's popl working on algebraic effects and CBPV.
- Read[5], a paper which looks to make a generalization for continuations and algebraic effects in the concurrent setting. They build a framework for understanding this pattern that shows up in a bunch of concurrent functional programming contexts (and also with church numerals(!). There could be a project looking to find of implement this in the wild
- Read [6], another paper interested in CBPV, this time looking to do a bunch of category theory to include both Call by Name and Call by Value interpretation into one category or something like that. The issue is that

these operations break associativity, so they use Duploids (what are these?) to solve their problems.
- Read [7], a pretty type-theory focused paper which tries to reflect some structure from comprehension categories back into the type systems they support. They offer some prohects to extend their work, tryign to show their syntax is complete (?) and trying to compare their work to bicategorical type theory.
- Read [8], a paper which looks to build monoidal structures that can describe everything we see in CS that looks like a monoid but doesnt work out. In particular they are interested in generalizing relative monads (whatever those are) and CBPV . Once again we get a discussion of associativity being broken by CBN and CBV. They solve this with bi-skew multicategories. It seems like their is a project to compare all these different ways of grappling with how CBPV breaks things. Maybe some research could emerge from that. This paper is also pretty generous with future work. Syntactic treatment of their monoid, trying to work on a theorem for representing bi-skew multicategories, and exploring what monoid actions look like (maybe in a multi*act*agory - whatever that means).

3.2. **What I wrote this week.** Brief summaries of the papers referenced above, some ideas for projects that we can discuss in our next meeting.

3.3. **Questions.** I tried very hard not to dive into every definition and calculation in these papers, so I have questions about pretty much every programming language and category theoretic term referenced above.

3.4. **Next week plan.** Before our next meeting (M-TH), continue lit review, focusing slightly more on abstract interpretation review.

In our meeting: decide between available project options (will discuss them in as much detail as I am able then) .

After our meeting: look for "tentpole" papers and allow a bit more time to dive into common definitions (and WRITING the common definitions).

References

[1] K. Ghorbal, E. Goubault, and S. Putot, "A logical product approach to zonotope intersection," in *Proceedings of the 22nd International Conference on Computer Aided Verification*, ser. CAV'10, Edinburgh, UK: Springer-Verlag, 2010, pp. 212–226, ISBN: 364214294X. DOI: 10.1007/978-3-642-14295-6_22 [Online]. Available: https://doi.org/10.1007/978-3-642-14295-6_22

[2] K. Ghorbal, E. Goubault, and S. Putot, "The zonotope abstract domain taylor1+," in *Computer Aided Verification*, A. Bouajjani and O. Maler, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 627–633, ISBN: 978-3-642-02658-4.

[3] R. Bagnara, P. M. Hill, E. Ricci, and E. Zaffanella, "Precise widening operators for convex polyhedra," *Science of Computer Programming*, vol. 58, no. 1, pp. 28–56, 2005, Special Issue on the Static Analysis Symposium 2003, ISSN: 0167-6423. DOI: https://doi.org/10.1016/j.scico.2005.02.003 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167642305000432

[4] E. Jones,
bibinitperiod Main, C. Li, J. Marriott, and
bibinitperiod Kavvos, "Domain-theoretic semantics for functional logic programming," English, *Proceedings of the ACM on Programming Languages*, vol. 10, no. POPL, pp. 1641–1672, Jan. 2026, Publisher Copyright: © 2026 Owner/Author.; 53rd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2026), POPL 2026 ; Conference date: 11-01-2026 Through 17-01-2026, ISSN: 2475-1421. DOI: 10.1145/3776699 [Online]. Available: https://popl26.sigplan.org/

[5] D. O. Kidney and N. Wu, "Hyperfunctions: Communicating continuations," *Proceedings of the ACM on Programming Languages*, vol. 10, pp. 174–203, 2026. [Online]. Available: https://api.semanticscholar.org/CorpusID:283885146

[6] É. Mangel, P.-A. Melliès, and G. Munch-Maccagnoni, "Classical notions of computation and the hasegawa-thielecke theorem," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026. DOI: 10.1145/3776715 [Online]. Available: https://doi.org/10.1145/3776715

[7] N. Najmaei, N. van der Weide, B. Ahrens, and P. R. North, "From semantics to syntax: A type theory for comprehension categories," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026. DOI: 10.1145/3776725 [Online]. Available: https://doi.org/10.1145/3776725

[8] P. B. Levy and M. Rogers, "What is a monoid?" *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026. DOI: 10.1145/3776727 [Online]. Available: https://doi.org/10.1145/3776727